

Lab Task 11: Working with ADO.NET using SqlCommand and SqlDataReader

Objective: Use SqlConnection, SqlCommand, and SqlDataReader to retrieve and insert data into the Employee table.

Tasks:

1. Retrieve Data (Read):

- Use SqlCommand.ExecuteReader() to select all employees where IsResigned = 0.
- Display the Name and Salary of each employee in the console.

2. Insert Data (Write):

- Use SqlCommand.ExecuteNonQuery() to insert a new employee into the Employee table.
- Use parameterized queries to pass values: Name, Salary, Department, IsResigned, and Experience.

Objective:

Use SqlDataAdapter and DataSet to read and update employee data in a disconnected environment.

Tasks:

1. Load Data:

- Use SqlDataAdapter.Fill() to load all employee data into a DataSet.
- Loop through the DataTable rows and display each employee's Name, Department, and Experience.

2. Update Data:

- Locate a specific employee by Name in the DataSet and increase their Experience by 1.
- Use SqlCommandBuilder and SqlDataAdapter.Update() to save the changes back to the database.

✓ Lab Task 12: Working with Entity Framework using DbContext

Objective: Use Entity Framework (Code First or Database First) to perform CRUD operations on the Employee table using DbContext.

Tasks:

1. Setup DbContext and Entity

Class Create an Employee entity class with properties:

- Id (int, primary key)
- Name (string)
- Salary (decimal)
- Department (string)
- IsResigned (bool)
- Experience (int)

Create an AppDbContext class inheriting from DbContext. Add a DbSet<Employee> property named Employees.

2. Retrieve Data (Read)

- Using LINQ with DbContext, select all employees where IsResigned = false.
- Display each employee's Name and Salary in the console.

3. Insert Data (Create)

- Create a new Employee object.
- Assign values for Name, Salary, Department, IsResigned, and Experience.
- Add the object to the context using context.Employees.Add().
- Save changes with context.SaveChanges().

4. Update Data (Update)

- Retrieve a specific employee by Name.
- Increase their Experience by 1.
- Save changes back to the database using context.SaveChanges().

5. Delete Data (Delete)

- Retrieve an employee by Id.
- Remove them using `context.Employees.Remove()`.
- Save changes with `context.SaveChanges()`.

Lab Task 13: Working with Session, TempData, and Cookies

Objective: Understand and implement state management in ASP.NET Core using Session, TempData, and Cookies.

Tasks:

1. Session

- Configure session in `Startup.cs` (or `Program.cs` in .NET 6/7+).
- Create a form to take user's Name and Email.
- Store the values in Session.
- Retrieve and display the stored session values on another page.
- Add a button to clear session

2. TempData

- Create an action that sets a TempData message (e.g., "Employee added successfully!").
- Redirect to another action/view where the message is displayed once.
- Demonstrate that TempData persists only for a single request.

3. Cookies

- Create a form to enter Theme Preference (e.g., Light or Dark).
- Store the preference in a Cookie with expiration of 1 day.
- Retrieve the cookie value and apply it to the page (e.g., change background color based on theme).
- Add a button to delete the cookie.

✓ Lab Task 14: Authentication and Authorization (with HttpContext)

Objective: Implement manual authentication (using cookies + HttpContext) and role-based authorization without ASP.NET Core Identity.

Tasks:

1. Authentication (Login/Logout with Cookies)

- Create a simple Login form with Username and Password.
- On successful login:
 - Create a ClaimsPrincipal with claims (Name, Role).
 - Use HttpContext.SignInAsync() to issue an authentication cookie.
- On logout:
 - Call HttpContext.SignOutAsync() to clear the cookie.

2. Authorization (Role-based)

- Use the [Authorize] attribute to protect controllers/actions.
- Restrict access:
 - [Authorize] → Any authenticated user.
 - [Authorize(Roles = "Admin")] → Only Admin users.
 - [Authorize(Roles = "User")] → Only User role.

3. Custom Policy Example

- Create a custom requirement: e.g., user must have claim "Experience" >= 2.
- Register a policy in Program.cs using AddAuthorization().
- Apply it using [Authorize(Policy = "RequireExperience")].

4. Demonstration Flow

- Login as **Admin** → Can access Admin-only pages.
- Login as **User** → Denied when accessing Admin pages.
- Apply **policy-based restriction** on experience → Users with less than 2 years cannot access.